

EFFICIENT COMPUTATION OF TRIPLE SCHUBERT STRUCTURE CONSTANTS

MATTHEW J. SAMUEL

ABSTRACT.

Keywords: Schubert polynomial, structure constant, Littlewood-Richardson rule, coproduct, double Schubert polynomial

Mathematics classification codes: 05E05, 05A05, 05A15, 14N10

Email: matthematics@gmail.com

1. INTRODUCTION

2. FOUNDATIONAL FORMULAS

2.1. Notation and definitions.

Definition 2.1 ($\mathbf{c}$, dominant permutations, θ , μ). We say that a permutation μ is *dominant* if $\mathbf{c}_i(\mu) \geq \mathbf{c}_{i+1}(\mu)$ for all i . For a permutation $v \in S_\infty$, we define a dominant permutation $\mu(v)$ recursively as follows. If v is already dominant, define $\mu(v) = v$. Otherwise, let i be the maximal index such that $\mathbf{c}_i(v) < \mathbf{c}_{i+1}(v)$, and define

$$\mu(v) = \mu(vs_i)$$

Define $\theta_i(v)$ to be $\mathbf{c}_i(\mu(v))$.

Definition 2.2 ($\xrightarrow{i}$, $\sigma_i(u, v)$, $D_i(u, v)$). Let $u, v \in S_\infty$ and $i \geq 1$ be an integer. We define $u \xrightarrow{i} v$ if there exist distinct integers $b_1, b_2, \dots, b_k$ such that $b_j \neq i$ for all j , $\ell(ut_{i,b_1} \cdots t_{i,b_k}) = \ell(u) + k$ for all k , and

$$v = ut_{i,b_1} \cdots t_{i,b_k}$$

We define

$$\sigma_i(u, v) = \#\{j \mid b_j < i\}$$

and we define

$$D_i(u, v) = \{u(i)\} \cup \{u(j) \mid u(j) \neq v(j)\}$$

2.2. The formula for double Schubert polynomials.

Theorem 2.3 (Formula for double Schubert polynomials). *Suppose $v \in S_\infty$. Then*

$$\mathfrak{S}_v(x; z) = \sum_{(v_0, \dots, v_k)} \prod_{i=1}^k (-1)^{\sigma_i(v_{i-1}, v_i)} E_{\theta_i(v) - \ell(v_{i-1}, v_i), \theta_i(v)}(x; z_{D_i(v_{i-1}, v_i)})$$

Where $v_{i-1} \xrightarrow{i} v_i$ for all i , $v_0 = 1$, and $v_k = v\mu(v^{-1})$.

2.3. The Pieri formula.

Theorem 2.4 (The Pieri formula, [7, Theorem 7.1]). *Suppose $u \in S_\infty$, $k \geq 1$, and $0 \leq p \leq k$. Let x, y, z be sets of variables. Then*

$$\mathfrak{S}_u(x; y) E_{p,k}(x; z) = \sum_{u \rightsquigarrow w} E_{p-\ell(u,w), k-\ell(u,w)}(y; z_{P_k(u,w)}) \mathfrak{S}_w(x; y)$$

where $P_k(u, w) = \{u(i) \mid i \leq k \text{ and } u(i) = w(i)\}$.

3. THE **pier**i ALGORITHM

Definition 3.1 (*k*-strip cover, admissible *k*-chain, the (p, k) -Pieri problem). Let $w \in S_n$, let $1 \leq k < n$, and let $\beta \in \{1, \dots, n\} \cup \{\infty\}$. A *k-strip cover* of (w, β) is a transposition t_{ab} of positions a, b with $1 \leq a \leq k < b \leq n$ such that

- (1) $w < wt_{ab}$ in Bruhat order; that is, $w(a) < w(b)$ and no c with $a < c < b$ satisfies $w(a) < w(c) < w(b)$; and
- (2) $w(b) < \beta$.

Its *output* is the pair $(wt_{ab}, w(b))$. An *admissible k-chain of length p* from u is a sequence $u = w_0, w_1, \dots, w_p$ carrying bounds $\infty = \beta_0 > \beta_1 > \dots > \beta_p$ in which (w_t, β_t) is the output of a *k-strip cover* of (w_{t-1}, β_{t-1}) for every t . The (p, k) -Pieri problem for u asks to enumerate every admissible *k-chain* from u of length at most p . These chains index the permutations occurring in $e_p(x_1, \dots, x_k) \mathfrak{S}_u$, where e_p is the elementary symmetric polynomial.

Method 3.2 (**pier**i(u, p, k)). Given $u \in S_n$, $1 \leq k < n$, and $0 \leq p \leq k$, the algorithm returns a list T of pairs (w, t) , one for each admissible *k-chain* from u of length $t \leq p$, recording its endpoint w .

- (1) Initialize $T \leftarrow [(u, 0)]$ and a frontier $F \leftarrow [(u, \infty)]$.
- (2) For $t = 1, \dots, p$, form a new frontier $F' \leftarrow []$ and, for each $(w, \beta) \in F$:
 - (a) compute $A \leftarrow \{a \in \{1, \dots, k\} \mid w(a) < \beta\}$;
 - (b) for each $b = k+1, \dots, n$ with $w(b) < \beta$, and each $a \in A$ for which t_{ab} is a *k-strip cover* of (w, β) , append $(wt_{ab}, w(b))$ to F' and (wt_{ab}, t) to T .
 Then set $F \leftarrow F'$.
- (3) Return T .

Lemma 3.3 (Multiplicity-freeness). *For each t , distinct admissible k -chains from u of length t have distinct endpoints; equivalently, the endpoint map is a bijection from admissible k -chains of length t onto the permutations w with $w \succ^k u$ of length $\ell(u) + t$ produced by the Pieri rule for $e_t(x_1, \dots, x_k)$.*

Proof. The strictly decreasing bounds $\beta_0 > \beta_1 > \dots$ force the values moved leftward across position k to be strictly decreasing, which is exactly the condition making the e_t -Pieri expansion multiplicity-free; see [1]. Reconstructing the chain from its endpoint by peeling off these moves in decreasing order of value shows the map is injective, hence bijective onto the Pieri support. $\square$

Proposition 3.4 (Complexity of **pier**i). *Let $M := |T|$ be the number of admissible k -chains from u of length at most p that **pier**i(u, p, k) returns. The algorithm runs in*

$$O(k(n-k)n \cdot M)$$

elementary operations, that is $O(n^3)$ per enumerated permutation. Moreover, by Lemma 3.3,

$$M \leq \#\{w \in S_n \mid w \geq u, \ell(w) \leq \ell(u) + p\}.$$

Proof. Processing one frontier node (w, β) costs $O(k)$ to build A , and then iterates over the $n - k$ choices of b and the at most k elements of A ; for each pair the Bruhat-cover test and the transposition each scan $O(n)$ positions. Thus a node costs $O(k + (n - k)kn) = O(k(n - k)n)$. By Lemma 3.3 the number of nodes ever placed on a frontier equals $\sum_{t=0}^{p-1} N_t$, where N_t is the number of length- t chains; since $\sum_{t=0}^{p-1} N_t \leq \sum_{t=0}^p N_t = M$, the total cost is $O(k(n - k)n \cdot M)$. The bound on M holds because every endpoint satisfies $w \geq u$ and $\ell(w) \leq \ell(u) + p$. $\square$

4. THE **elemrepr** ALGORITHM

We now describe an algorithm **elemrepr** for expressing a double Schubert polynomial as in Theorem 2.3. Its starting point is the structure of a *dominant* double Schubert polynomial. If μ is dominant with $\mathfrak{c}(\mu) = (\lambda_1, \dots, \lambda_m)$, then $\mathfrak{S}_\mu(x; z)$ factors as a product of factorial elementary symmetric polynomials of maximal degree, one per column of the shape,

$$\mathfrak{S}_\mu(x; z) = \prod_{i=1}^m E_{\lambda_i, \lambda_i}(x; z^{(i)}),$$

where $z^{(i)}$ is the shifted z -alphabet attached to column i [4]. For a general v one writes $\mathfrak{S}_v(x; z)$ by applying divided differences in the z -variables to the dominant polynomial $\mathfrak{S}_{\mu(v^{-1})}(x; z)$; because that polynomial factors by column, the divided-difference action decouples and may be carried out one column (one $z^{(i)}$ -group) at a time. Applied to a single column this produces a *signed* descending expansion. The following primitive carries it out.

Definition 4.1 (k -strip descent beneath an edge, the down-Pieri problem). Let $w, \mu \in S_n$ and $1 \leq k < n$. Say w' lies *beneath the edge* μ if

$$\text{inv}(w'\mu) = \text{inv}(\mu) - \text{inv}(w'),$$

that is, $\ell(w'\mu) = \ell(\mu) - \ell(w')$. A k -strip descent of w is a Bruhat descent $w \succ wt_{ab}$ realized by a transposition t_{ab} that moves the value in the pivot position k either leftward (from a position $a < k$, with sign -1) or rightward (to a position $b > k$, with sign $+1$), touching only positions still holding their original value. Given a starting label w , the permutation μ , a degree p , and the column k , the *down-Pieri problem* asks for the signed list of those length- t descending k -strip chains from w ($t \leq p$) whose endpoint lies beneath μ in Bruhat order.

Method 4.2 (**kdown**(w, μ, p, k)). The algorithm returns a signed list D of triples (w', t, s) , one for each length- t descending k -strip chain from w ($t \leq p$) whose endpoint w' lies beneath μ in Bruhat order, with $s \in \{+1, -1\}$ its sign.

- (1) Initialize $D \leftarrow []$; if w is already beneath μ , append $(w, 0, +1)$. Set a frontier $F \leftarrow [(w, +1)]$.
- (2) For $t = 1, \dots, p$, form $F' \leftarrow []$ and, for each $(w'', s) \in F$:
 - (a) (leftward) for each position $a < k$ with $w''(a) = w(a)$ such that $w'' \succ w''t_{a,k}$ is a Bruhat descent, set $w''' \leftarrow w''t_{a,k}$ and append $(w''', -s)$ to F' ; if w''' is beneath μ , append $(w''', t, -s)$ to D ;
 - (b) (rightward) for each position $b > k$ with $w''(b) = w(b)$ such that $w'' \succ w''t_{k,b}$ is a Bruhat descent, set $w''' \leftarrow w''t_{k,b}$ and append $(w''', +s)$ to F' ; if w''' is beneath μ , append $(w''', t, +s)$ to D .
 Then set $F \leftarrow F'$.
- (3) Return D .

Applied without the edge μ , the signed descending expansion produces many extraneous terms that ultimately cancel. The edge is exactly the bound that discards them: only endpoints beneath μ are retained in D . In the assembled v -side (Method 4.3 below) only these retained labels seed the next column, so the edge prevents the number of live paths from blowing up across columns.

Method 4.3 (**elemrepr**(v), the v -side path dictionaries). Let $\theta := \theta(v^{-1})$ with nonzero parts in columns $1, \dots, L$, and let $vmu := v\mu(v^{-1})$. Processing columns from $i = L$ down to $i = 1$, maintain for each column a set of live labels (initialized at column L to the single label vmu). At column i , with $k = i$ and edge $\mu_i := \mu$ of the shape truncated below column i , call **kdown**(w, μ_i, θ_i, k) for each live label w ; record its signed output triples as the transitions out of w , and let their endpoints be the live labels seeding column $i - 1$. The collected transitions form the path dictionaries used by **schubmult**.

Lemma 4.4 (Correctness of the edge filter). *A permutation w' produced by a descending k -strip chain from w is retained by **kdown**(w, μ, p, k) if and only if w' lies beneath μ , i.e. $\text{inv}(w'\mu) = \text{inv}(\mu) - \text{inv}(w')$. In the assembly of Method 4.3 the retained signed labels are exactly the terms of $\mathfrak{S}_v(x; z)$ contributed through column i , so no extraneous (cancelling) label propagates to column $i - 1$.*

Proof. The retention test is the identity $\text{inv}(w'\mu) = \text{inv}(\mu) - \text{inv}(w')$ checked in step (2); this is precisely the condition that $w' \leq \mu$ with $\ell(w'\mu) = \ell(\mu) - \ell(w')$. The factorization of the dominant polynomial by column, together with the column-wise decoupling of the z -divided differences, identifies the beneath- μ endpoints at column i with the surviving monomials of $\mathfrak{S}_v(x; z)$ after processing that column; the extraneous chains are exactly those leaving the shape, which the edge discards before they seed column $i - 1$. $\square$

Proposition 4.5 (Complexity of **kdown**/**elemrepr**). *Let $R := |D|$ be the number of retained signed triples and F^* the total number of frontier nodes generated by **kdown**(w, μ, p, k). The algorithm runs in*

$$O(n^2 \cdot F^*)$$

elementary operations, and $R \leq F^$. Consequently **elemrepr**(v), which invokes **kdown** once per live label across the $L \leq \min(n - 1, \ell_v)$ columns and by Lemma 4.4 propagates only retained labels, runs in $O(n^2 \cdot \Phi_v)$ operations, where Φ_v is the total number of frontier nodes generated over all columns.*

Proof. Processing one frontier node scans $O(n)$ candidate positions, each requiring an $O(n)$ Bruhat-descent test and $O(n)$ transposition, so $O(n^2)$ per node; summing over the F^* nodes gives $O(n^2 F^*)$. Each retained triple arises from a distinct frontier node, so $R \leq F^*$. In **elemrepr**, by Lemma 4.4 the labels seeding column $i - 1$ are only the retained (beneath-edge) endpoints of column i , so the live-label count never includes extraneous paths; summing the per-column frontier work gives $O(n^2 \Phi_v)$. $\square$

5. THE **schubmult** ALGORITHM

Method 5.1 (The algorithm **schubmult**). Fix n and let $u, v \in S_n$. The algorithm **schubmult**(u, v) computes the expansion of $\mathfrak{S}_u(x; y) \mathfrak{S}_v(x; z)$ in the Schubert basis as follows. Set $\theta := \theta(v^{-1})$ and let

$$L := \#\{i \mid \theta_i \neq 0\}$$

be the number of nonzero parts.

The arithmetic takes place in the coefficient ring $R := \mathbb{Z}[y; z]$, in which the structure constants live. The algorithm proceeds in L layers and maintains a hash H that maps each pair (w, λ) —an intermediate u -side permutation w together with a v -side label λ —to a coefficient $c(w, \lambda) \in R$. All coefficients are initialized to 0 except for the single key (u, e) , which is initialized to $c(u, e) = 1$. Write $\mu := \mu(v^{-1})$ and $vmu := v\mu$.

Each layer transforms H by one column of the shape. Fix layer i and put $k := \theta_i$. For a key (w, λ) carrying a nonzero coefficient $c = c(w, \lambda)$:

- (1) Call **pieri**(w, θ_i, θ_i) (Method 3.2) to enumerate the u -side covers $w \rightsquigarrow w'$; each such cover carries a degree $a \in \{0, 1, \dots, \theta_i\}$, the number of boxes it adds in column i .
- (2) Read from **elemrepr**(v) (Method 4.3) the v -side transitions out of λ in column i : each is a signed triple (λ', b, s) with endpoint λ' , degree b , and sign $s \in \{+1, -1\}$.
- (3) For every pair (cover $w \rightsquigarrow w'$ of degree a , triple (λ', b, s)) with $a + b \leq \theta_i$, accumulate into the new hash the single ring element

$$c(w', \lambda') \ += \ s \cdot c \cdot E_{\theta_i - a - b, \theta_i - a}(y_A; z_B),$$

where $E_{p,m}(y; z)$ denotes the factorial elementary symmetric polynomial of degree p in the first m variables,

$$E_{p,m}(y; z) := \sum_{1 \leq i_1 < \dots < i_p \leq m} \prod_{j=1}^p (y_{i_j} - z_{i_j - j + 1}),$$

evaluated on the y -alphabet $y_A := \{y_{w'(j)} \mid j \leq \theta_i, w(j) = w'(j)\}$ of the $\theta_i - a$ values the cover leaves fixed in columns $1, \dots, \theta_i$, and the z -alphabet z_B recorded by the v -side descent $\lambda \rightsquigarrow \lambda'$.

Thus each layer performs, for every matched (cover, triple) pair, the construction of one factorial elementary symmetric polynomial $E_{p,m}(y; z)$, followed by one ring multiplication (by the sign and the running coefficient) and one ring addition into the accumulator $c(w', \lambda')$. The polynomial $E_{p,m}(y; z)$ is rebuilt on each pair by the divide-and-conquer recursion of Lemma 5.2: this recursion is not memoized in the y -alphabet—only the z -index selection is cached—so its construction is a nontrivial cost, recorded as E_n in Definition 5.4, and not a shared lookup. Every coefficient $c(w, \lambda)$ is kept in factored form—a product of the factorial elementary symmetric factors $E_{p,m}(y; z)$ accumulated along its paths—and is never multiplied out into the monomial basis, so it stays far smaller than its $\binom{n}{2}$ -degree monomial expansion, whose number of terms is exponential in n . After all L layers the coefficient $c(w, vmu)$ at each u -side endpoint w is the structure constant of $\mathfrak{S}_w(x; y)$ in $\mathfrak{S}_u(x; y) \mathfrak{S}_v(x; z)$.

Lemma 5.2 (Divide-and-conquer splitting of $E_{p,m}$). *Let $y = (y_1, \dots, y_m)$ and let $z = (z_r)_{r \in \mathbb{Z}}$ be an alphabet. For an integer c put*

$$E_{p,m}^{(c)}(y; z) := \sum_{1 \leq i_1 < \dots < i_p \leq m} \prod_{j=1}^p (y_{i_j} - z_{i_j - j + 1 + c}),$$

so that $E_{p,m} = E_{p,m}^{(0)}$. Set $t := \lfloor m/2 \rfloor$, $y' := (y_1, \dots, y_t)$, and $y'' := (y_{t+1}, \dots, y_m)$. Then for $0 \leq p \leq m$ and every integer c ,

$$E_{p,m}^{(c)}(y; z) = \sum_{a=0}^p E_{a,t}^{(c)}(y'; z) E_{p-a, m-t}^{(c+t-a)}(y''; z),$$

with the conventions $E_{0,\ell}^{(c)} = 1$ and $E_{a,\ell}^{(c)} = 0$ for $a > \ell$, and the base cases $E_{1,\ell}^{(c)}(y; z) = \sum_{i=1}^{\ell} (y_i - z_{i+c})$ and $E_{\ell,\ell}^{(c)}(y; z) = \prod_{i=1}^{\ell} (y_i - z_{1+c})$.

Proof. Classify each index set $1 \leq i_1 < \dots < i_p \leq m$ in the defining sum of $E_{p,m}^{(c)}$ by the number a of its members lying in $\{1, \dots, t\}$; the remaining $p-a$ lie in $\{t+1, \dots, m\}$. The first a factors $\prod_{j=1}^a (y_{i_j} - z_{i_j-j+1+c})$ range over all size- a subsets of $\{1, \dots, t\}$ and sum to $E_{a,t}^{(c)}(y'; z)$. For the upper part write $i_{a+r} = t + l_r$ with $1 \leq l_1 < \dots < l_{p-a} \leq m-t$; the corresponding factor is

$$y_{t+l_r} - z_{i_{a+r}-(a+r)+1+c} = y''_{l_r} - z_{l_r-r+1+(c+t-a)},$$

which is the r -th factor of $E_{p-a, m-t}^{(c+t-a)}(y''; z)$. So the upper factors sum to $E_{p-a, m-t}^{(c+t-a)}(y''; z)$; the terms with $a > t$ contribute 0, consistent with the convention. Summing over a gives the identity. $\square$

Lemma 5.3 (Output support in S_{2n-1}). *Let $w_0 = w_0(n)$ be the longest element of S_n , with $\mathbf{c}(w_0) = (n-1, n-2, \dots, 1, 0)$, and let $w_1 = w_1(n)$ be the permutation with*

$$\mathbf{c}(w_1) = \mathbf{c}(w_0) + \mathbf{c}(w_0) = (2n-2, 2n-4, \dots, 2, 0),$$

a dominant element of S_{2n-1} with $\ell(w_1) = 2\binom{n}{2}$. For any $u, v \in S_n$, every permutation w occurring in the product $\mathfrak{S}_u \cdot \mathfrak{S}_v$ satisfies $w \leq w_1$; in particular $w \in S_{2n-1}$.

Proof. Both $u \leq w_0$ and $v \leq w_0$ in Bruhat order. Standard bounds for products of Schubert polynomials with dominant cofactors place the support of $\mathfrak{S}_u \cdot \mathfrak{S}_v$ weakly below the dominant permutation whose code is $\mathbf{c}(w_0) + \mathbf{c}(w_0)$, namely w_1 . Since $\mathbf{c}(w_1) = (2n-2, \dots, 2, 0)$ has largest part $2n-2$, we have $w_1(1) = 2n-1$, so $w_1 \in S_{2n-1}$; hence every $w \leq w_1$ also lies in S_{2n-1} . $\square$

Definition 5.4 ($\ell_u, \ell_v, P_u, B, \Delta, E_n$). For $u, v \in S_n$ write $\ell_u := \ell(u)$ and $\ell_v := \ell(v)$, and set

$$P_u := \#\{w \in S_{2n-1} \mid w \geq u \text{ and } \ell(w) \leq \ell_u + \ell_v\}.$$

By Lemma 5.3 every intermediate and output u -side permutation is counted by P_u , and $P_u \leq (2n-1)!$. Write $\theta := \theta(v^{-1})$. The two *per-layer branchings* are

$$B := \max_i \max_w |\mathbf{pieri}(w, \theta_i, \theta_i)|, \quad \Delta := \max_i \max_{\lambda} \#\{\text{kdown triples } (\lambda', c, \varepsilon) \text{ carried by } \lambda \text{ in column } i\} :$$

B is the maximum number of u -side covers generated by a single **pieri** step (Proposition 3.4), and Δ is the maximum number of v -side kdown triples generated from a single label (Proposition 4.5). Each branching enumerates a family of k -strips indexed by (nonempty) subsets of at most n eligible positions, so

$$B \leq 2^n - 1, \quad \Delta \leq 2^n - 1;$$

the bound on B is sharp, attained at $w = w_0$ (where **pieri**($w_0, n-1, n$) produces exactly $2^n - 1$ covers).

Finally, let $R(p, m)$ denote the number of coefficient-ring operations (additions and multiplications) performed by the unmemoized divide-and-conquer recursion of Lemma 5.2 in evaluating $E_{p,m}^{(c)}(y; z)$; the count is independent of the offset c . With $t := \lfloor m/2 \rfloor$ it satisfies $R(p, m) = 0$ for $p = 0$ or $p > m$, the base values $R(1, \ell) = R(\ell, \ell) = \ell - 1$, and, for $1 < p < m$,

$$R(p, m) = R(p, t) + R(p, m-t) + 1 + \sum_{a=\max(1, p-m+t)}^{\min(p, t)} (R(a, t) + R(p-a, m-t) + 2).$$

Define

$$E_n := \max_{0 \leq p \leq m \leq n-1} R(p, m).$$

Proposition 5.5. $\log_2 E_n = \Theta(\log^2 n)$; equivalently $E_n = 2^{\Theta(\log^2 n)} = n^{\Theta(\log n)}$, so E_n is superpolynomial in n but subexponential.

Proof. Write $C(m) := \max_{0 \leq p \leq m} R(p, m)$, so $E_n = \max_{m \leq n-1} C(m)$. Throughout put $t := \lfloor m/2 \rfloor$, so the two child sizes t and $m-t$ lie in $\{\lfloor m/2 \rfloor, \lceil m/2 \rceil\}$, and let N be the number of admissible indices a in the recurrence, so $N \leq t + 1 \leq m$.

Upper bound. For $1 < p < m$ the recurrence has $2N + 2$ summands of the form $R(\cdot, t)$ or $R(\cdot, m - t)$, each at most $C(\lceil m/2 \rceil)$, together with $2N + 1$ additive constants; hence, using $C(\lceil m/2 \rceil) \geq 1$ for $m \geq 3$,

$$R(p, m) \leq (2N + 2) C(\lceil m/2 \rceil) + (2N + 1) \leq (4m + 3) C(\lceil m/2 \rceil).$$

Since $C(2) = 1$, induction on m gives $C(m) \leq (4m + 3)^{\lceil \log_2 m \rceil}$: the step uses $4\lceil m/2 \rceil + 3 \leq 4m + 3$ and $1 + \lceil \log_2 \lceil m/2 \rceil \rceil = \lceil \log_2 m \rceil$. Therefore

$$\log_2 C(m) \leq \lceil \log_2 m \rceil \log_2(4m + 3) = O(\log^2 m),$$

and so $\log_2 E_n = O(\log^2 n)$.

Lower bound. Let $I(p, m)$ count the internal nodes—calls with $1 < p < m$ —of the unmemoized recursion tree of $R(p, m)$. Each internal node performs at least one ring addition, so $R(p, m) \geq I(p, m)$, and I satisfies, for $1 < p < m$,

$$I(p, m) = 1 + I(p, t) + I(p, m - t) + \sum_{a=\max(1, p-m+t)}^{\min(p, t)} (I(a, t) + I(p - a, m - t)).$$

Put $J(m) := \sum_{p=0}^m I(p, m)$. Dropping all terms of the sum except $I(a, t)$, a fixed value $a^* \in \{1, \dots, t\}$ is an admissible index in the recurrence for $R(p, m)$ precisely when $a^* \leq p \leq a^* + (m - t)$, which holds for at least $\lceil m/2 \rceil$ values of p with $1 < p < m$; summing the resulting contribution $I(a^*, t)$ over those p gives

$$J(m) \geq \left\lfloor \frac{m}{2} \right\rfloor \sum_{a^*=1}^t I(a^*, t) = \left\lfloor \frac{m}{2} \right\rfloor J(t).$$

As $J(4) \geq 1$ (since $I(2, 4) \geq 1$), unrolling this inequality down to a fixed constant size yields

$$\log_2 J(m) \geq \sum_{i=0}^{\lceil \log_2 m \rceil - 3} \log_2 \left\lfloor \frac{m}{2^{i+1}} \right\rfloor = \Omega(\log^2 m).$$

Finally $J(m) \leq (m + 1) \max_p I(p, m) \leq (m + 1) C(m)$, so $\log_2 C(m) \geq \log_2 J(m) - \log_2(m + 1) = \Omega(\log^2 m)$, whence $\log_2 E_n \geq \log_2 C(n - 1) = \Omega(\log^2 n)$. $\square$

Theorem 5.6 (Complexity of **schubmult**). *The number of coefficient-ring operations performed by **schubmult**(u, v) is*

$$O(\min(n - 1, \ell_v) \cdot P_u \cdot 2^{\ell_v} \cdot B \cdot \Delta \cdot E_n),$$

where B , Δ , and E_n are as in Definition 5.4. Writing this bound as $2^{f(n)}$, the worst case is

$$f(n) = \binom{n}{2} + \log_2((2n - 1)!) + 2n + \Theta(\log^2 n) + O(\log n),$$

and, when v is dominant, the bound is $2^{\log_2((2n-1)!) + n + \Theta(\log^2 n)}$.

Remark 5.7. Here B and Δ are the u -side (Pieri) and v -side (kdown-triple) per-layer branchings and E_n is the per-iteration elementary-symmetric construction cost. Each additive term of $f(n)$ is a multiplicative factor of the running time, so all are retained. The dominant term is $\binom{n}{2}$, whence the worst-case running time is $2^{(1+o(1))\binom{n}{2}}$; the remaining summands are of strictly lower order in the exponent, the largest being $\log_2((2n - 1)!) = 2n \log_2 n + O(n)$.

Proof. By Method 5.1 the algorithm runs in $L = \#\{i \mid \theta_i(v^{-1}) \neq 0\} \leq \min(n - 1, \ell_v)$ layers. Write $\mu := \mu(v^{-1})$, so that $\sum_i \theta_i(v^{-1}) = \ell(\mu)$; note that $\ell(\mu) \geq \ell_v$, with equality if and only if v is dominant, so the total degree processed on the shape is in general strictly larger than ℓ_v . The key point is that the two sides are yoked: at layer i a u -side degree u_i and a v -side degree v_i are paired with $u_i + v_i = \theta_i$, and the v -side descends from $v\mu$ to the identity, so $\sum_i v_i = \ell(v\mu)$. Hence the total degree added on the u -side is

$$\sum_i u_i = \sum_i \theta_i - \sum_i v_i = \ell(\mu) - \ell(v\mu) = \ell_v,$$

using $\ell(v\mu) = \ell(\mu) - \ell(v)$ (as $v^{-1} \leq \mu$ in weak order). Thus every intermediate u -side permutation satisfies $w \geq u$ and $\ell(w) \leq \ell_u + \ell_v$; by Lemma 5.3 it also satisfies $w \leq w_1(n)$, hence lies in S_{2n-1} , so there are at most P_u of them. The v -side labels lie in the lower interval $[e, v\mu]$; since $\ell(v\mu) = \ell(\mu)$ can exceed ℓ_v , we bound

their number by $2^{\ell(\mu)}$, but the edge filter of Lemma 4.4 retains only those beneath the running dominant edge, of which there are at most 2^{ℓ_v} live at once. Hence the hash at the start of each layer has at most $P_u \cdot 2^{\ell_v}$ keys.

We now account for the work. Write Φ_i for the set of keys (w, λ) present in H at the start of layer i , each a u -side permutation w paired with a v -side label λ . The inner-loop iteration of the algorithm matches one **pieri** cover of w against one kdown triple of λ , and is executed precisely

$$W = \sum_{i=1}^L \sum_{(w, \lambda) \in \Phi_i} \beta_i(w) \delta_i(\lambda)$$

times, where $\beta_i(w) := |\mathbf{pieri}(w, \theta_i, \theta_i)|$ is the u -side Pieri branching of Proposition 3.4 and $\delta_i(\lambda)$ is the number of kdown triples of Proposition 4.5 carried by λ in column i . This is a product: at every key the v -side kdown branching $\delta_i(\lambda)$ multiplies the u -side Pieri branching $\beta_i(w)$, so the signed kdown factor enters the running time as a multiplicative branching rather than a lower-order additive term. Bounding each factor by its maximum, $\beta_i(w) \leq B$ and $\delta_i(\lambda) \leq \Delta$, and the key count by $|\Phi_i| \leq P_u \cdot 2^{\ell_v}$, gives

$$W \leq L \cdot P_u \cdot 2^{\ell_v} \cdot B \cdot \Delta$$

iterations. Each executed iteration constructs one factorial elementary symmetric polynomial $E_{p,m}(y; z)$ (with $p \leq m \leq \theta_i \leq n-1$) by the divide-and-conquer recursion of Lemma 5.2. Because the recursion is not memoized in the y -alphabet—only the z -index selection is cached—this is a fresh evaluation costing E_n ring operations (Definition 5.4), not a lookup; the ensuing multiplication by the sign and the running (factored) coefficient, the ring addition into the accumulator, and the $O(n)$ permutation bookkeeping are all lower-order. Since $E_n = \omega(n)$, the total number of ring operations is

$$T = O(W \cdot E_n) = O(L \cdot P_u \cdot 2^{\ell_v} \cdot B \cdot \Delta \cdot E_n),$$

which is the bound of Theorem 5.6.

For the worst case take base-2 logarithms and substitute $L \leq n$, $P_u \leq (2n-1)!$ (Lemma 5.3), $\ell_v \leq \binom{n}{2}$, $B, \Delta \leq 2^n - 1$, and $\log_2 E_n = \Theta(\log^2 n)$ (Proposition 5.5):

$$\log_2 T \leq \binom{n}{2} + \log_2((2n-1)!) + 2n + \Theta(\log^2 n) + O(\log n).$$

We retain every summand: an additive term in the exponent is a multiplicative factor of T —the two branchings alone contribute $B\Delta \leq 4^n$, i.e. the $+2n$, the permutation count contributes $\log_2((2n-1)!)$, and the elementary-symmetric construction contributes $E_n = 2^{\Theta(\log^2 n)}$ —so none may be discarded. Only at the very end do we simplify $\log_2((2n-1)!) = (2n-1)\log_2(2n-1) - O(n) = 2n\log_2 n + O(n)$; as this is $o(n^2)$, the dominant term is $\binom{n}{2} = \Theta(n^2)$ and $T = 2^{(1+o(1))\binom{n}{2}}$. If v is dominant then $\theta(v) = \mathbf{c}(v)$, $\ell(\mu) = \ell_v$, every path step carries a $+1$ sign, and each layer's v -side is a single path, so $\Delta = 1$ and only $O(\ell_v)$ labels are live in place of 2^{ℓ_v} ; then $T = O(L \cdot P_u \cdot \ell_v \cdot B \cdot E_n)$ and $\log_2 T \leq \log_2((2n-1)!) + n + \Theta(\log^2 n) + O(\log n)$, i.e. $T = 2^{\log_2((2n-1)!) + n + \Theta(\log^2 n)} = 2^{2n\log_2 n + O(n) + \Theta(\log^2 n)}$. $\square$

Corollary 5.8 (Ordinary Schubert polynomials). *For ordinary (single) Schubert polynomials, computing the structure constants of the product $\mathfrak{S}_u(x) \mathfrak{S}_v(x)$ by the single-variable specialization of **schubmult** takes*

$$O(\min(n-1, \ell_v) \cdot P_u \cdot 2^{\ell_v} \cdot B \cdot \Delta)$$

integer operations. Writing this bound as $2^{g(n)}$, the worst case is

$$g(n) = \binom{n}{2} + \log_2((2n-1)!) + 2n + O(\log n),$$

and, when v is dominant, the bound is $2^{\log_2((2n-1)!) + n + O(\log n)}$. These are the bounds of Theorem 5.6 with the term $\Theta(\log^2 n)$ deleted.

Proof. The single-variable algorithm has the same combinatorial skeleton as Method 5.1: the shape $\theta := \theta(v^{-1})$, the u -side Pieri covers, and the v -side path structure with its signed transitions (λ', b, s) are identical, so the hash, the number of layers $L \leq \min(n-1, \ell_v)$, and the iteration count $W \leq L \cdot P_u \cdot 2^{\ell_v} \cdot B \cdot \Delta$ are exactly as in the proof of Theorem 5.6. The single difference is the weight attached to a matched (cover, triple) pair: for double Schubert polynomials it is the factorial elementary symmetric polynomial $E_{p,m}(y; z)$,

whereas for ordinary Schubert polynomials the corresponding Pieri coefficient is 1, so the pair contributes only the integer sign $s \in \{+1, -1\}$. Each executed iteration therefore performs one integer multiplication by s and one integer addition into the accumulator— $O(1)$ ring operations—so E_n is replaced by $O(1)$ and $T = O(W)$. Taking base-2 logarithms and substituting the same bounds $L \leq n$, $P_u \leq (2n-1)!$ (Lemma 5.3), $\ell_v \leq \binom{n}{2}$, $B, \Delta \leq 2^n - 1$ (and $\Delta = 1$ in the dominant case) gives $g(n)$, which is $f(n)$ of Theorem 5.6 without the $\Theta(\log^2 n)$ contribution of E_n . $\square$

Remark 5.9 (Comparison with prior complexity results). The complexity of a single structure constant has been studied more than that of an entire product. For Schur polynomials the coefficients $c_{\lambda\mu}^\nu$ are the Littlewood–Richardson coefficients, and computing one of them is $\#P$ -complete [5]. Since these coefficients can be exponentially large, a single one has polynomial bit-length yet cannot be evaluated in time polynomial in the binary sizes of the input and output unless $FP = \#P$; the same then holds for the full product $s_\lambda s_\mu = \sum_\nu c_{\lambda\mu}^\nu s_\nu$. The Littlewood–Richardson rule instead enumerates skew tableaux, of which there are $\sum_\nu c_{\lambda\mu}^\nu$, and its running time is polynomial in that number, namely in the coefficients written in unary. Deciding whether a given coefficient is positive is, by contrast, in P , through the saturation theorem [3] and the reduction to hive-polytope nonemptiness [2].

For Schubert polynomials the coefficients c_{uv}^w generalize the Littlewood–Richardson coefficients, recovered in the Grassmannian case, so computing one is $\#P$ -hard; whether they lie in $\#P$ is open, no combinatorial rule being known. The vanishing problem $\{c_{uv}^w = 0\}$ was recently placed in the polynomial hierarchy, in *coAM* assuming the generalized Riemann hypothesis, hence in Σ_2^P [6].

Since a single coefficient is already $\#P$ -hard, neither the bound of Theorem 5.6 nor that of Corollary 5.8 is polynomial in the binary output size, and the two are compared instead per combinatorial witness. Each executed iteration of **schubmult** processes one matched (cover, triple) pair. Corollary 5.8 performs $O(1)$ integer operations at each such pair, so in the single case the number of ring operations is proportional to the number of witnesses. Theorem 5.6 performs $E_n = n^{\Theta(\log n)}$ operations at each pair (Proposition 5.5), a quasi-polynomial factor—superpolynomial but subexponential in n —incurred by rebuilding the factorial elementary symmetric weight; the single specialization removes it because that weight collapses to a sign.

REFERENCES

- [1] BERGERON, N., AND SOTTILE, F. Skew Schubert functions and the Pieri formula for flag manifolds. *Trans. Amer. Math. Soc.* 354, 2 (2002), 651–673.
- [2] BÜRGISSE, P., AND IKENMEYER, C. Deciding positivity of Littlewood–Richardson coefficients. *SIAM Journal on Discrete Mathematics* 27, 4 (2013), 1639–1681.
- [3] KNUTSON, A., AND TAO, T. The honeycomb model of $GL_n(\mathbb{C})$ tensor products I: Proof of the saturation conjecture. *Journal of the American Mathematical Society* 12, 4 (1999), 1055–1090.
- [4] MACDONALD, I. G. *Notes on Schubert Polynomials*. Université Du Québec à Montréal, Dép. de mathématiques et d’informatique, 1991.
- [5] NARAYANAN, H. On the complexity of computing Kostka numbers and Littlewood–Richardson coefficients. *J. Algebraic Combin.* 24, 3 (2006), 347–354.
- [6] PAK, I., AND ROBICHAUX, C. Vanishing of Schubert coefficients. *arXiv preprint arXiv:2412.02064* (2024). with an appendix joint with David E. Speyer.
- [7] SAMUEL, M. J. A Molev–Sagan type formula for double Schubert polynomials. *J. Pure Appl. Algebra* 228, 7 (2024), 107636.